

## Classic Problems of Synchronization

### The Readers-Writers Problem

- A database is to be shared among several concurrent processes.
- Some of these processes may want **only to read the database**, whereas **others may want to update** (that is, to read and write) the database.
- We distinguish between these two types of processes by referring to the former as **Readers** and to the latter as **Writers**.
- Obviously, if two readers access the shared data simultaneously, no adverse affects will result.
- However, if a writer and some other thread (either a reader or a writer) access the database simultaneously, chaos may ensue.

**To ensure that these difficulties do not arise, we require that the writers have exclusive access to the shared database.**

**This synchronization problem is referred to as the readers-writers problem.**

### Solution to the Readers-Writers Problem using Semaphores:

We will make use of **two semaphores** and an **integer variable**:

1. **mutex**, a semaphore (initialized to 1) which is used to ensure mutual exclusion when **readcount** is updated i.e. when any reader enters or exit from the critical section.
2. **wrt**, a semaphore (initialized to 1) common to both reader and writer processes.
3. **readcount**, an integer variable (initialized to 0) that keeps track of how many processes are currently reading the object.

## Process Implementation (Pseudo-code)

### Writer Process

The writer must ensure exclusive access before performing a write operation.

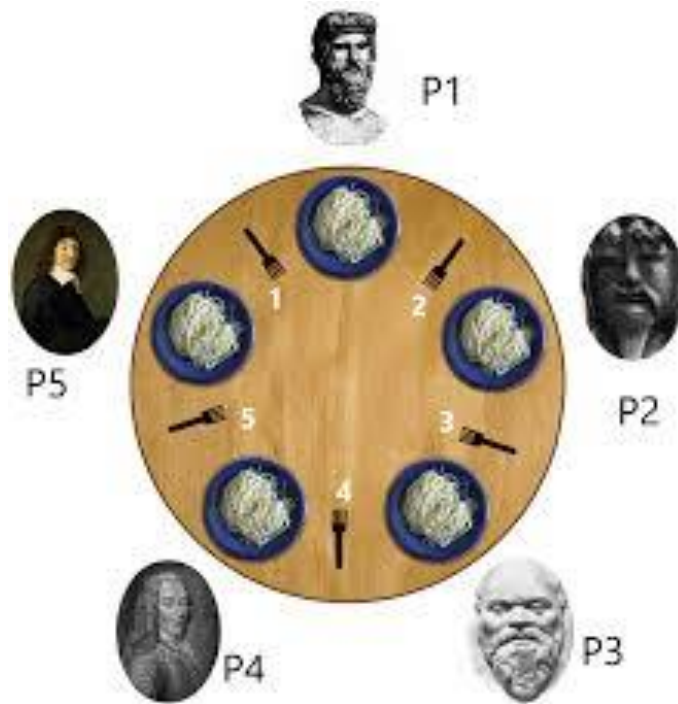
```
do {  
    /* writer requests for critical  
    section */  
  
    wait(wrt);  
  
    /* performs the write */  
  
    // leaves the critical section  
    signal(wrt);  
  
} while(true);
```

### Reader Process

Readers allow multiple concurrent reads but must block writers if at least one reader is active.

```
do {  
    wait(mutex);  
  
    readcnt++; // The number of readers has now increased by 1  
  
    if (readcnt == 1)  
        wait(wrt); // ensures no writer can enter if there is even one  
        reader  
  
    signal(mutex); // other readers can enter while this current reader is  
    inside  
  
    /* current reader performs reading here */  
  
    wait(mutex);  
  
    readcnt--; // a reader wants to leave  
  
    if (readcnt == 0) // no reader is left in the critical section  
        signal(wrt); // writers can enter  
  
    signal(mutex); // reader leaves  
  
} while(true);
```

## The Dining-Philosophers Problem



### Philosopher either

- **Thinks**
  - When a philosopher thinks, he does not interact with his colleagues
- **Eats**
  - When a philosopher gets hungry he tries to pick up the two forks that are closest to him (left & right). A philosopher may pick up only one fork at a time.

One cannot pick up a fork that is already in the hand of a neighbor.

When a hungry philosopher has both his forks at the same time, he eats without releasing his forks.

When he has finished eating, he puts down both of his forks and starts thinking again.

### A Simple Solution Using Semaphores

- One simple solution is to represent each fork/chopstick with a **semaphore**.
- A philosopher tries to grab a fork/chopstick by executing a **wait()** operation on that semaphore.

- He releases his fork/chopsticks by executing the **signal()** operation on the appropriate semaphores.
- Thus, the shared data are:
  - semaphore chopstick [5];

where all the elements of chopstick are initialized to **1**.

The structure of **philosophers i**

```
do {
    wait (chopstick [i] );
    wait (chopstick [ (i + 1) % 5] );
    ....
    // eat
    signal(chopstick [i]);
    signal(chopstick [ (i + 1) % 5]);
    // think
} while (TRUE);
```

**Now,**

- Although this solution guarantees that no two neighbors are eating simultaneously, it could still create a **deadlock**.
- Suppose that all five philosophers become hungry simultaneously and each grabs their left chopstick.
- All the elements of chopstick will now be equal to **0**.
- When each philosopher tries to grab his right chopstick, he will be **delayed forever**.

**Some Possible Remedies to Avoid Deadlocks**

- Allow at most four philosophers to be sitting simultaneously at the table.

- Allow a philosopher to pick up his chopsticks only if both chopsticks are available (to do this he must pick them up in a critical section).
- Use an asymmetric solution; that is, an odd philosopher picks up first his left chopstick and then his right chopstick, whereas an even philosopher picks up his right chopstick and then his left chopstick.

## Monitor

The biggest problem with semaphores is that the programmer must control everything manually, so mistakes are very easy.

**Semaphore = Traffic light with NO controller**

- You must:
  - Turn red ON
  - Turn green OFF
  - Do it in the right order
- If you forget **one step** → accident

That's semaphore.

**Monitor = Automatic gate with guard**

- You just enter
- Guard handles:
  - Locking
  - Waiting
  - Waking up
- You **can't forget** anything

That's monitor.

## Problems with Semaphores

- **Incorrect use of semaphore operations:**
  - `signal(mutex) ... wait(mutex)`
  - `wait(mutex) ... wait(mutex)`
- **Using them incorrectly may result in deadlocks, timing errors etc. that are difficult to detect.**

| <b>Semaphore</b> | <b>Monitor</b>    |
|------------------|-------------------|
| Manual control   | Automatic control |
| Error-prone      | Safer             |
| Hard to read     | Easy to read      |
| Low-level        | High-level        |

- **Semaphore** → Do everything yourself
- **Monitor** → System takes care

**Monitor: High-level abstraction** - provides a convenient and effective mechanism for process synchronization

- **Abstract data type:**
  - Encapsulates data with a set of functions to operate on that data
  - A set of programmer-defined operations provided with mutual exclusion within monitor
  - **Declares variables**
    - Values define state of an instance of that type
    - Can be accessed by only local functions
  - **Functions that operate on variables**
    - Can access only those variables declared locally within monitor
    - Can access its formal parameters

## Syntax of Monitors:

```
monitor monitor_name
{
    // shared variable declarations
    procedure P1 (...) {
        ...
    }
    procedure P2 (...) {
        ...
    }
    .
    .
    procedure Pn (...) {
        ...
    }
    initialization code (...) {
        ...
    }
}
```

## Key Characteristics

- A procedure defined within a monitor can access only those variables declared locally within the monitor and its formal parameters.
- Similarly, the local variables of a monitor can be accessed by only the local procedures.
- **The monitor construct ensures that only one process at a time can be active within the monitor.**

## Condition Construct

**condition x, y;**

The only operations that can be invoked on a condition variable are **wait()** and **signal()**.

- The operation **x.wait()**; means that the process invoking this operation is suspended until another process invokes **x.signal()**;
- The **x.signal()** operation resumes exactly one suspended process.

**Condition construct is needed in a monitor so a process can WAIT safely when it cannot continue, without blocking others.**

**Why is it needed?**

**Monitor alone only gives:**

- **Mutual exclusion** (one process at a time)

But sometimes:

- A process **enters the monitor**
- The resource is **not ready**
- It **must wait**

What should it do?

**Without condition construct**

**Option 1: Busy waiting**

```
while (buffer empty) {  
    // keep checking  
}
```

Wastes CPU

Other processes can't enter

System becomes slow

**Option 2: Hold monitor while waiting**

- Process stays inside
- Others cannot enter
- Condition never changes

Deadlock



## Using Condition Construct

Condition construct lets a process:

Go to sleep

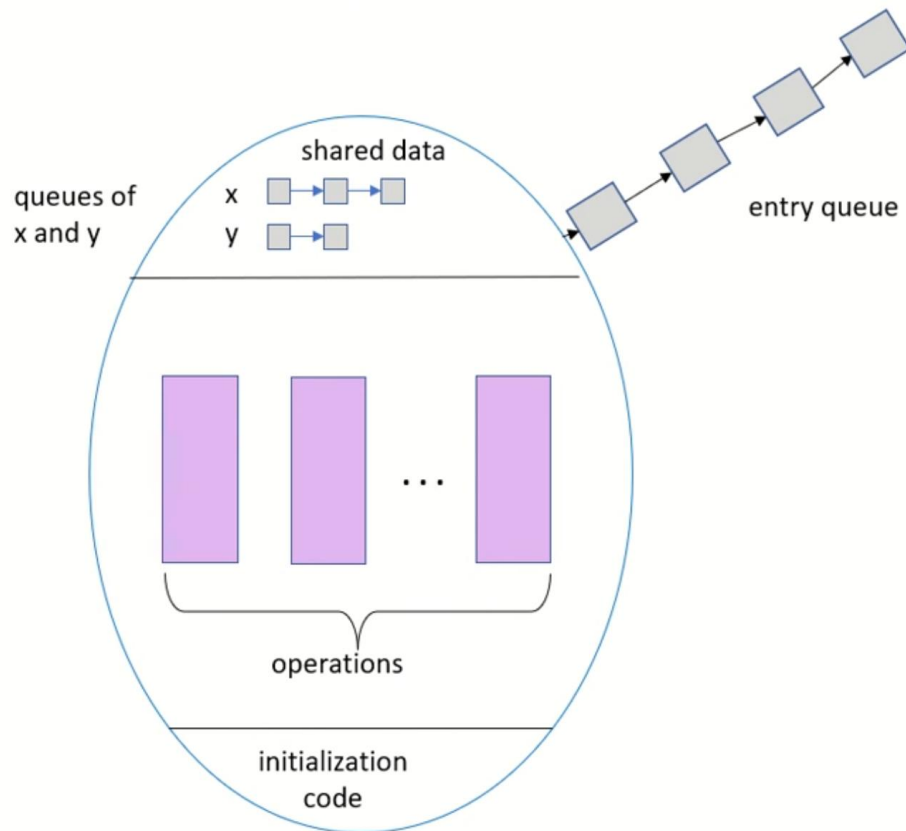
Release the monitor

Wait for a signal

Wake up and try again

- Monitor = one-at-a-time
- Condition construct = wait politely

## Schematic View of a Monitor:



## Dining-Philosophers Solution Using Monitors

- We now illustrate monitor concepts by presenting a deadlock-free solution to the dining-philosophers problem.
- This solution imposes the **restriction** that a philosopher may **pick up his chopsticks** only if **both of them are available**.
- To code this solution, we need to distinguish among three states in which we may find a philosopher. For this purpose, we introduce the following data structure:

**enum { thinking, hungry, eating } state [5];**

- **Philosopher i** can set the variable **state[i] = eating** only if his two neighbors are not eating: (state [(i+4) % 5] != eating) and (state [(i+1)% 5] != eating).
- We also need to declare **condition self [5];** where philosopher i can delay himself when he is hungry but is unable to obtain the chopsticks he needs

### Code:

```
monitor dp {
    enum { THINKING, HUNGRY, EATING } state [5];
    condition self [5];

    void pickup (int i) {
        state [i] = HUNGRY;
        test (i);
        if (state [i] != EATING)
            self [i].wait();
    }

    void putdown(int i) {
        state [i] = THINKING;
        test ((i + 4) % 5);
        test ((i + 1) % 5);
    }

    initialization-code () {
        for (int i = 0; i < 5; i++)
            state [i] = THINKING;
    }
}
```

```
void test (int i) {
    if ((state [(i + 4) % 5] !=
        EATING) &&
        (state [i] == HUNGRY)
        &&
        (state [(i + 1) % 5] !=
        EATING)) {
        state [i] = EATING;
        self [i].signal();
    }
}
```